PHASE 1 OF 8

Project Overview & Vision

What EduGenie is, who it serves, and why it is built as a hybrid cloud + local AI study assistant.

Purpose | Core Features | Audience | Design Goals

1.1 What is EduGenie?

EduGenie is a lightweight web application that acts as an **AI study assistant**. A student opens it in a browser and can ask any question, request a simple explanation of a concept, paste in a passage to get a short summary, turn a passage into a multiple-choice quiz, or ask for a step-by-step learning path on a topic.

Behind the scenes, EduGenie is a small **FastAPI** backend that forwards requests to either Google's **Gemini API** (Q&A, summaries, quizzes and learning paths) or a **locally-run open-source model** (plain-language explanations). This hybrid design keeps the app fast and cost-efficient: the Explain feature works entirely offline once the local model is cached, while the more open-ended features benefit from Gemini's larger, cloud-hosted model.

1.2 Core Features

Feature	What the student does	Powered by	Internet?
Q&A	Types any question and receives a written answer.	Gemini (cloud)	Required
Explain	Enters a concept and gets a simplified, school-level explanation.	LaMini-Flan-T5-783M (local)	Only for first download
Summarize	Pastes a passage and receives a plain-language summary.	Gemini (cloud)	Required
Quiz	Supplies a passage or topic and gets 3 auto-graded multiple-choice questions.	Gemini (cloud)	Required
Learning Path	Names a topic and receives a beginner → advanced roadmap with resources.	Gemini (cloud)	Required

At a glance

Five features, one FastAPI backend, two AI engines. The browser never talks to Google directly — every request goes through the backend, which keeps the API key private and the frontend simple.

1.3 The Problem EduGenie Solves

Students who use AI tools for study often juggle several sites: one to ask questions, another to summarize notes, another to practise with quizzes. Each tool has its own interface, and none is tuned for *learning* specifically. EduGenie brings the most common study actions into a single, minimal page so that the path from "I don't understand this" to "I can test myself on it" takes only a few clicks.

Goals

- **One place to study** — ask, explain, summarize, quiz and plan without switching tools.
- **Simple by default** — plain HTML forms and a stateless backend that a new contributor can read in an afternoon.
- **Cost- and latency-aware** — use a small local model where a big cloud model is unnecessary.
- **Easy to extend** — one module per feature, one route per module.
- **Safe by design** — secrets stay on the server; user input is validated before it reaches a model.

1.4 Why a Hybrid Design?

Splitting the work between a cloud model and a local model is a deliberate trade-off rather than an inconsistency. The table below summarizes how each engine fits the features it serves.

Consideration	Gemini (cloud)	LaMini-Flan-T5 (local)
Best suited for	Open-ended reasoning: Q&A, summaries, quiz JSON, multi-step learning paths	A single-purpose, latency-sensitive task: short plain-language explanations
Model size / quality	Large, frequently updated	Small (783M parameters), concise output
Network needed	Yes, on every call	No, once weights are cached
Cost profile	Metered API usage	Free per request; uses local CPU/RAM
First-run cost	None beyond API key	One-time download of a few hundred MB

1.5 Design Principles in Practice

Principle	How it shows up in the codebase
Separation of concerns	Routes live in `main.py`; prompts and model calls live in feature modules.
Fail gracefully	Missing keys and malformed model output return friendly messages instead of crashing.
Load once, reuse	The local model and tokenizer are cached in memory after the first request.
Keep secrets server-side	The Gemini key is read from `.env` by the backend only.

> **Success looks like**
> A student can go from a pasted passage to a graded quiz in under a minute, with no account, no setup on their side, and no confusing errors when something goes wrong.

1.6 Scope of This Version

In scope	Out of scope (today)
Five study features exposed as HTTP routes and a single-page UI	User accounts, login and authentication
Stateless request handling — each call is independent	Saving quiz scores or learning-path history
Input validation (non-empty fields, 400 errors)	Automated test suite
Local model caching after first download	Request timeouts and rate limiting
Auto-graded quiz in the browser	Multi-user deployment hardening (CORS, auth)

1.7 Typical User Journeys

Scenario	Steps	Features used
Revising for a test	Paste chapter notes → read the summary → generate a 3-question quiz → check answers.	Summarize, Quiz
Meeting a new concept	Type the concept → read the simple explanation → ask a follow-up question.	Explain, Q&A
Starting a new subject	Enter the topic → receive a leveled roadmap with videos, articles and books.	Learning Path
Studying offline	Use Explain with cached weights when there is no connectivity.	Explain

1.8 How This Documentation Set Is Organized

The original single document has been divided into eight self-contained phases. Read them in order for a complete onboarding, or jump straight to the phase you need.

Phase	Title	What you will find
1	Project Overview & Vision	Purpose, features, scope (this document)
2	Architecture & Project Structure	Layers, request lifecycle, file layout
3	Requirements & Environment Setup	Prerequisites, install steps, configuration files
4	Backend Core & Gemini Modules	main.py, qna.py, summary_module.py, learning_path.py
5	Local Model & Quiz Engine	explanation_module.py, quiz_module.py, check_models.py
6	API Reference & User Interface	Endpoints, curl examples, UI walkthrough, data flows
7	Testing, Troubleshooting & FAQ	Test plan, fixes for common errors, questions answered
8	Security, Roadmap & Conclusion	Known gaps, hardening, future work, appendices